

[← Back to Blog](#)[Python](#) [FastAPI](#) [A2A Protocol](#) [AI](#) [JSON-RPC](#)

Building A2A Protocol Agents with Python and FastAPI

Phoenix • October 26, 2025 • 15 min read

Building A2A Protocol Agents with Python and FastAPI

The Agent-to-Agent (A2A) protocol is revolutionizing how AI agents communicate across different platforms. While frameworks like Mastra provide excellent TypeScript implementations, Python developers have equally powerful tools at their disposal. In this guide, I'll walk you through building a production-ready A2A agent using Python and FastAPI.

Why Python for A2A Agents?

Python has become the de facto language for AI and machine learning, offering:

- **Rich AI Ecosystem:** Libraries like Pydantic AI, LangChain, and transformers
- **FastAPI Performance:** Async/await support with near-native speed
- **Type Safety:** Modern Python with Pydantic for robust data validation
- **Easy Integration:** Simple connection to ML models, databases, and external APIs

What is the A2A Protocol?

The A2A protocol is a standardized communication layer based on JSON-RPC 2.0 that allows AI agents to interact seamlessly. It defines:

- Message format and structure
- Task lifecycle management

- Artifact handling and history tracking
- Error handling and status reporting

Project Setup

Let's start by setting up our Python project. We'll use modern Python 3.13+ with FastAPI and related libraries.

1. Initialize the Project

bash



```
# Create project directory
mkdir chess-agent-a2a
cd chess-agent-a2a

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Initialize pyproject.toml
touch pyproject.toml
```

2. Configure Dependencies

Create your `pyproject.toml` :

toml



```
[project]
name = "chess-agent-a2a"
version = "0.1.0"
description = "A chess-playing agent with A2A protocol support"
requires-python = "≥3.13"
dependencies = [
    "fastapi[all] ≥0.115.12",
    "pydantic-ai ≥0.4.2",
    "httpx ≥0.28.1",
    "python-dotenv ≥1.1.0",
    "redis[hiredis] ≥6.0.0",
    "jsonrpcclient ≥4.0.3",
    "chess ≥1.11.2",
    "cairosvg ≥2.7.1",
    "minio ≥7.2.15"
]

[build-system]
```

```
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

3. Install Dependencies

bash



```
pip install -e .
```

Building the Core A2A Server

1. Define A2A Message Models

First, let's create Pydantic models for the A2A protocol:

python



```
# models/a2a.py
from pydantic import BaseModel, Field
from typing import Literal, Optional, List, Dict, Any
from datetime import datetime
from uuid import uuid4

class MessagePart(BaseModel):
    kind: Literal["text", "data", "file"]
    text: Optional[str] = None
    data: Optional[Dict[str, Any]] = None
    file_url: Optional[str] = None

class A2AMessage(BaseModel):
    kind: Literal["message"] = "message"
    role: Literal["user", "agent", "system"]
    parts: List[MessagePart]
    messageId: str = Field(default_factory=lambda: str(uuid4()))
    taskId: Optional[str] = None
    metadata: Optional[Dict[str, Any]] = None

class PushNotificationConfig(BaseModel):
    url: str
    token: Optional[str] = None
    authentication: Optional[Dict[str, Any]] = None

class MessageConfiguration(BaseModel):
    blocking: bool = True
    acceptedOutputModes: List[str] = ["text/plain", "image/png", "image/svg+xml"]
    pushNotificationConfig: Optional[PushNotificationConfig] = None

class MessageParams(BaseModel):
```

```

message: A2AMessage
configuration: MessageConfiguration = Field(default_factory=MessageConfiguration)

class ExecuteParams(BaseModel):
    contextId: Optional[str] = None
    taskId: Optional[str] = None
    messages: List[A2AMessage]

class JSONRPCRequest(BaseModel):
    jsonrpc: Literal["2.0"]
    id: str
    method: Literal["message/send", "execute"]
    params: MessageParams | ExecuteParams

class TaskStatus(BaseModel):
    state: Literal["working", "completed", "input-required", "failed"]
    timestamp: str = Field(default_factory=lambda: datetime.utcnow().isoformat())
    message: Optional[A2AMessage] = None

class Artifact(BaseModel):
    artifactId: str = Field(default_factory=lambda: str(uuid4()))
    name: str
    parts: List[MessagePart]

class TaskResult(BaseModel):
    id: str
    contextId: str
    status: TaskStatus
    artifacts: List[Artifact] = []
    history: List[A2AMessage] = []
    kind: Literal["task"] = "task"

class JSONRPCResponse(BaseModel):
    jsonrpc: Literal["2.0"] = "2.0"
    id: str
    result: Optional[TaskResult] = None
    error: Optional[Dict[str, Any]] = None

```

2. Create the FastAPI Application

python



```

# main.py
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse
from contextlib import asynccontextmanager
from dotenv import load_dotenv
import os

from models.a2a import JSONRPCRequest, JSONRPCResponse, TaskResult, TaskStatus, Artifact, MessagePart, A2A
from agents.chess_agent import ChessAgent

load_dotenv()

# Initialize chess agent

```

```

chess_agent = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Lifespan context manager for startup and shutdown"""
    global chess_agent

    # Startup: Initialize the chess agent
    chess_agent = ChessAgent(
        engine_path=os.getenv("CHESS_ENGINE_PATH", "/usr/games/stockfish"),
        minio_endpoint=os.getenv("MINIO_ENDPOINT"),
        minio_access_key=os.getenv("MINIO_ACCESS_KEY"),
        minio_secret_key=os.getenv("MINIO_SECRET_KEY")
    )

    yield

    # Shutdown: Cleanup
    if chess_agent:
        await chess_agent.cleanup()

app = FastAPI(
    title="Chess Agent A2A",
    description="A chess-playing agent with A2A protocol support",
    version="1.0.0",
    lifespan=lifespan
)

@app.post("/a2a/chess")
async def a2a_endpoint(request: Request):
    """Main A2A endpoint for chess agent"""
    try:
        # Parse request body
        body = await request.json()

        # Validate JSON-RPC request
        if body.get("jsonrpc") != "2.0" or "id" not in body:
            return JSONResponse(
                status_code=400,
                content={
                    "jsonrpc": "2.0",
                    "id": body.get("id"),
                    "error": {
                        "code": -32600,
                        "message": "Invalid Request: jsonrpc must be '2.0' and id is required"
                    }
                }
            )

        rpc_request = JSONRPCRequest(**body)

        # Extract messages
        messages = []
        context_id = None
        task_id = None
        config = None

        if rpc_request.method == "message/send":
            messages = [rpc_request.params.message]
            config = rpc_request.params.configuration
        elif rpc_request.method == "execute":

```

```

        messages = rpc_request.params.messages
        context_id = rpc_request.params.contextId
        task_id = rpc_request.params.taskId

    # Process with chess agent
    result = await chess_agent.process_messages(
        messages=messages,
        context_id=context_id,
        task_id=task_id,
        config=config
    )

    # Build response
    response = JSONRPCResponse(
        id=rpc_request.id,
        result=result
    )

    return response.model_dump()

except Exception as e:
    return JSONResponse(
        status_code=500,
        content={
            "jsonrpc": "2.0",
            "id": body.get("id") if "body" in locals() else None,
            "error": {
                "code": -32603,
                "message": "Internal error",
                "data": {"details": str(e)}
            }
        }
    )

@app.get("/health")
async def health_check():
    """Health check endpoint"""
    return {"status": "healthy", "agent": "chess"}

if __name__ == "__main__":
    import uvicorn
    port = int(os.getenv("PORT", 5001))
    uvicorn.run(app, host="0.0.0.0", port=port)

```

3. Implement the Chess Agent

python



```

# agents/chess_agent.py
import chess
import chess.svg
from uuid import uuid4
from typing import List, Optional
from datetime import datetime
import asyncio

```

```

from io import BytesIO
import cairosvg
from minio import Minio

from models.a2a import (
    A2AMessage, TaskResult, TaskStatus, Artifact,
    MessagePart, MessageConfiguration
)

class ChessAgent:
    def __init__(
        self,
        engine_path: str,
        minio_endpoint: str,
        minio_access_key: str,
        minio_secret_key: str
    ):
        self.engine_path = engine_path
        self.boards = {} # Store game states by context_id

        # Initialize MinIO client for storing board images
        self.minio_client = Minio(
            minio_endpoint,
            access_key=minio_access_key,
            secret_key=minio_secret_key,
            secure=False
        )

        # Ensure bucket exists
        if not self.minio_client.bucket_exists("chess-boards"):
            self.minio_client.make_bucket("chess-boards")

    async def process_messages(
        self,
        messages: List[A2AMessage],
        context_id: Optional[str] = None,
        task_id: Optional[str] = None,
        config: Optional[MessageConfiguration] = None
    ) → TaskResult:
        """Process incoming messages and generate chess moves"""

        # Generate IDs if not provided
        context_id = context_id or str(uuid4())
        task_id = task_id or str(uuid4())

        # Get or create board for this context
        board = self.boards.get(context_id, chess.Board())

        # Extract last user message
        user_message = messages[-1] if messages else None
        if not user_message:
            raise ValueError("No message provided")

        # Extract move from message
        move_text = ""
        for part in user_message.parts:
            if part.kind == "text":
                move_text = part.text.strip()
                break

        # Apply user's move

```

```

try:
    move = board.parse_san(move_text)
    board.push(move)
except Exception as e:
    raise ValueError(f"Invalid move: {move_text}")

# Save updated board state
self.boards[context_id] = board

# Generate AI move using Stockfish
ai_move = await self._get_stockfish_move(board)
if ai_move:
    board.push(ai_move)
    ai_move_san = board.san(ai_move)
else:
    ai_move_san = "No legal moves available"

# Generate board visualization
board_svg = chess.svg.board(board)
board_url = await self._upload_board_image(board_svg, context_id, task_id)

# Build response message
response_text = f"I played {ai_move_san}"
if board.is_checkmate():
    response_text += " - Checkmate!"
elif board.is_check():
    response_text += " - Check!"

response_message = A2AMessage(
    role="agent",
    parts=[MessagePart(kind="text", text=response_text)],
    taskId=task_id
)

# Build artifacts
artifacts = [
    Artifact(
        name="move",
        parts=[MessagePart(kind="text", text=ai_move_san)]
    ),
    Artifact(
        name="board",
        parts=[MessagePart(kind="file", file_url=board_url)]
    )
]

# Build history
history = messages + [response_message]

# Determine state
state = "input-required" if not board.is_game_over() else "completed"

return TaskResult(
    id=task_id,
    contextId=context_id,
    status=TaskStatus(
        state=state,
        message=response_message
    ),
    artifacts=artifacts,
    history=history

```



```

    )

    async def _get_stockfish_move(self, board: chess.Board) → Optional[chess.Move]:
        """Get best move from Stockfish engine"""
        try:
            process = await asyncio.create_subprocess_exec(
                self.engine_path,
                stdin=asyncio.subprocess.PIPE,
                stdout=asyncio.subprocess.PIPE,
                stderr=asyncio.subprocess.PIPE
            )

            # Send UCI commands
            commands = [
                "uci\n",
                "isready\n",
                f"position fen {board.fen()}\n",
                "go movetime 1000\n"
            ]

            stdout, _ = await process.communicate("".join(commands).encode())
            output = stdout.decode()

            # Parse best move
            for line in output.split("\n"):
                if line.startswith("bestmove"):
                    move_uci = line.split()[1]
                    return chess.Move.from_uci(move_uci)

            return None
        except Exception as e:
            print(f"Stockfish error: {e}")
            # Fallback to random legal move
            legal_moves = list(board.legal_moves)
            return legal_moves[0] if legal_moves else None

    async def _upload_board_image(
        self,
        svg_content: str,
        context_id: str,
        task_id: str
    ) → str:
        """Upload board image to MinIO and return URL"""
        try:
            # Convert SVG to PNG
            png_data = cairosvg.svg2png(bytes(svg_content.encode()))

            # Upload to MinIO
            object_name = f"{context_id}/{task_id}.png"
            self.minio_client.put_object(
                "chess-boards",
                object_name,
                BytesIO(png_data),
                len(png_data),
                content_type="image/png"
            )

            # Return public URL
            return f"http://{self.minio_client._base_url.netloc}/chess-boards/{object_name}"
        except Exception as e:
            print(f"Image upload error: {e}")

```

```
        return ""

    async def cleanup(self):
        """Cleanup resources"""
        self.boards.clear()
```

Environment Configuration

Create a `.env` file:

bash



```
# Server Configuration
PORT=5001
DEPLOYMENT_TYPE=blocking # or "webhook"

# Chess Engine
CHESS_ENGINE_PATH=/usr/games/stockfish

# MinIO/S3 Configuration
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin

# Optional: Redis for session storage
REDIS_URL=redis://localhost:6379
```

Running the Agent

1. Start Required Services

bash

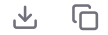


```
# Start MinIO (using Docker)
docker run -d \
  -p 9000:9000 \
  -p 9001:9001 \
  --name minio \
  -e "MINIO_ROOT_USER=minioadmin" \
  -e "MINIO_ROOT_PASSWORD=minioadmin" \
  quay.io/minio/minio server /data --console-address ":9001"

# Install Stockfish
sudo apt-get install stockfish # Ubuntu/Debian
# or
brew install stockfish # macOS
```

2. Run the FastAPI Server

bash



```
python main.py
```

Your A2A agent is now running at `http://localhost:5001` !

Testing the A2A Endpoint

Test with cURL

bash



```
curl -X POST http://localhost:5001/a2a/chess \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "id": "test-001",
  "method": "message/send",
  "params": {
    "message": {
      "kind": "message",
      "role": "user",
      "parts": [
        {
          "kind": "text",
          "text": "e4"
        }
      ]
    },
    "messageId": "msg-001",
    "taskId": "task-001"
  },
  "configuration": {
    "blocking": true
  }
}
```

Expected Response

json



```
{
  "jsonrpc": "2.0",
```

```

{id": "test-001",
"result": {
  "id": "task-001",
  "contextId": "generated-context-id",
  "status": {
    "state": "input-required",
    "timestamp": "2025-10-26T10:30:00.000Z",
    "message": {
      "messageId": "msg-uuid",
      "role": "agent",
      "parts": [
        {
          "kind": "text",
          "text": "I played e5"
        }
      ],
      "kind": "message",
      "taskId": "task-001"
    }
  },
  "artifacts": [
    {
      "artifactId": "artifact-uuid",
      "name": "move",
      "parts": [
        {
          "kind": "text",
          "text": "e5"
        }
      ]
    },
    {
      "artifactId": "artifact-uuid-2",
      "name": "board",
      "parts": [
        {
          "kind": "file",
          "file_url": "http://localhost:9000/chess-boards/context-id/task-001.png"
        }
      ]
    }
  ],
  "history": [...],
  "kind": "task"
}
}

```

Integration with Telex

Once your Python A2A agent is deployed, you can integrate it with Telex just like any other A2A agent:

json



```

{
  "nodes": [

```

```
{
  "id": "chess_agent",
  "name": "Chess Player",
  "type": "a2a/generic-a2a-node",
  "url": "https://your-server.com/a2a/chess"
}
```

Advanced Features

1. Webhook Support for Async Processing

For long-running operations, implement webhook notifications:

python



```
async def send_webhook_notification(
    webhook_url: str,
    result: TaskResult,
    auth: Optional[Dict[str, Any]] = None
):
    """Send result to webhook URL"""
    headers = {"Content-Type": "application/json"}

    if auth and auth.get("schemes") == ["TelexApiKey"]:
        headers["Authorization"] = f"Bearer {auth.get('credentials')}"

    async with httpx.AsyncClient() as client:
        await client.post(
            webhook_url,
            json=result.model_dump(),
            headers=headers
        )
```

2. Redis Session Storage

Store conversation state in Redis:

python



```
import redis.asyncio as redis

class SessionStore:
    def __init__(self, redis_url: str):
        self.redis = redis.from_url(redis_url)
```

```
async def save_board(self, context_id: str, board: chess.Board):
    await self.redis.set(
        f"chess:board:{context_id}",
        board.fen(),
        ex=3600 # 1 hour expiry
    )

async def load_board(self, context_id: str) → Optional[chess.Board]:
    fen = await self.redis.get(f"chess:board:{context_id}")
    return chess.Board(fen.decode()) if fen else None
```

3. Error Handling and Validation

Implement robust error handling:

python



```
from enum import Enum

class A2AErrorCode(Enum):
    PARSE_ERROR = -32700
    INVALID_REQUEST = -32600
    METHOD_NOT_FOUND = -32601
    INVALID_PARAMS = -32602
    INTERNAL_ERROR = -32603

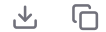
def create_error_response(
    request_id: str,
    code: A2AErrorCode,
    message: str,
    data: Optional[Dict] = None
):
    return {
        "jsonrpc": "2.0",
        "id": request_id,
        "error": {
            "code": code.value,
            "message": message,
            "data": data or {}
        }
    }
```

Deployment

Deploy to Production

1. Containerize with Docker:

dockerfile



```
FROM python:3.13-slim

# Install system dependencies
RUN apt-get update && apt-get install -y \
    stockfish \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY pyproject.toml .
RUN pip install -e .

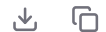
COPY . .

EXPOSE 5001

CMD ["python", "main.py"]
```

2. Deploy to Cloud Platform:

bash



```
# Build and push
docker build -t chess-agent-a2a .
docker push your-registry/chess-agent-a2a

# Deploy to your platform (Railway, Render, AWS, etc.)
```

Key Takeaways

1. **Python + FastAPI is Perfect for A2A:** Modern async support, type safety, and rich AI ecosystem
2. **Pydantic Models Ensure Type Safety:** Validate all A2A messages at runtime
3. **Stateful Agents Need Session Storage:** Use Redis or databases for conversation context
4. **Artifacts Enable Rich Responses:** Return both text and visual/data artifacts
5. **JSON-RPC 2.0 Compliance is Critical:** Follow the spec for interoperability

Comparison: Python vs TypeScript for A2A



Feature	Python + FastAPI	TypeScript + Mastra
Type Safety	✓ Pydantic	✓ Native TypeScript
AI Libraries	✓ ✓ Rich ecosystem	⚠ Growing
Performance	✓ Async/await	✓ Node.js async
Learning Curve	✓ Gentle	⚠ Moderate
Deployment	✓ Many options	✓ Mastra Cloud

Next Steps

Build your own A2A agents:

- **Translation Agent:** Multi-language support with OpenAI
- **Data Analysis Agent:** Process datasets with Pandas
- **Vision Agent:** Image analysis with computer vision
- **RAG Agent:** Knowledge retrieval with vector databases

The A2A protocol opens up endless possibilities. Whether you choose Python or TypeScript, you're building the future of agent communication!

Resources:

- [FastAPI Documentation](#)
- [A2A Protocol Specification](#)
- [Chess Agent Example](#)
- [Pydantic AI](#)

Built with Python 3.13, FastAPI 0.115, and deployed October 2025.

PHOENIX